
Padrões Estruturais

Adapter & Decorator



Prof. Dr. João Choma

UEM

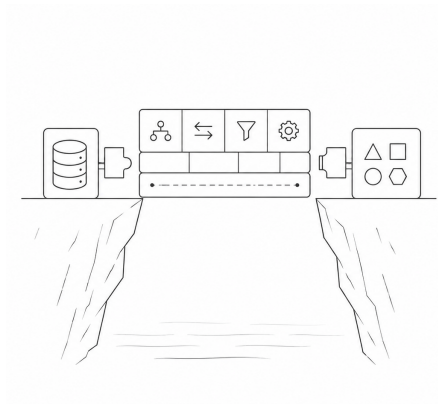
github.com/JoaoChoma

Fundamentos



Começaremos pelo problema, usaremos analogias e chegaremos ao Java.

- 1 Por que padrões estruturais existem?
- 2 Adapter: interfaces incompatíveis.
- 3 Decorator: comportamentos em camadas.
- 4 Implementações testáveis e extensíveis.





Eles aparecem quando o problema não é criar o objeto, mas **organizar a ligação entre objetos**.

- Uma API externa usa nomes diferentes do domínio.
- Um serviço precisa de log, cache, retry ou validação sem duplicação.
- Trocar uma dependência espalha mudanças por várias classes.

Padrão estrutural

Solução para compor, adaptar ou envolver objetos mantendo baixo acoplamento.



Categoria	Pergunta
Criacionais	Como criar objetos?
Estruturais	Como encaixar ou compor objetos?
Comportamentais	Como objetos interagem?

Adapter

Cria uma ponte entre interfaces incompatíveis.

Decorator

Envolve objetos mantendo a interface e acrescentando responsabilidades.

Adapter



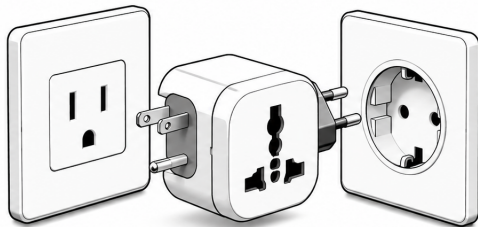
Analogia: tomada de viagem

O carregador funciona, mas seu formato não encaixa na tomada do país visitado.

- O carregador não precisa ser refeito.
- A tomada não precisa mudar.
- O adaptador traduz o encaixe.

No código

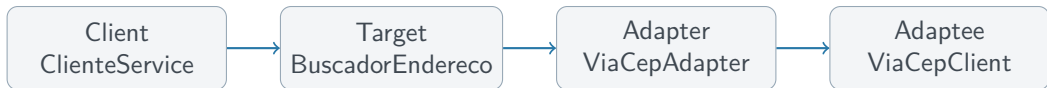
Uma classe espera uma interface, mas a classe existente oferece outra.





```
class ClienteService {  
    private ViaCepClient viaCep;  
  
    Endereco cadastrar(String cep) {  
        ViaCepDTO dto = viaCep.buscar(cep);  
        return new Endereco(  
            dto.logradouro(),  
            dto.localidade(),  
            dto.uf());  
    }  
}
```

- O domínio conhece ViaCepClient e ViaCepDTO.
- Trocar o fornecedor exige modificar o domínio.
- Testes precisam simular detalhes da API externa.



- Client usa apenas a interface do domínio.
- Target é o contrato que a aplicação deseja.
- Adapter traduz chamadas e dados.
- Adaptee é a classe externa ou incompatível.

Regra de ouro

O domínio depende do Target, não do Adapter concreto.



```
public interface BuscadorDeEndereco {  
    Endereco buscar(String cep);  
}  
  
public record Endereco(  
    String rua,  
    String cidade,  
    String estado  
) { }
```

O Target usa o vocabulário da aplicação e não expõe DTOs do fornecedor externo.



```
public class ViaCepClient {  
    public ViaCepDTO consultar(String cep) {  
        // chamada HTTP omitida  
        return new ViaCepDTO(  
            "Av. Brasil", "Maringa", "PR");  
    }  
}  
  
public record ViaCepDTO(  
    String logradouro,  
    String localidade,  
    String uf  
) { }
```

O Adaptee funciona, mas sua interface não é o contrato desejado pelo domínio.



```
public class ViaCepAdapter implements BuscadorDeEndereco {
    private final ViaCepClient client;

    public ViaCepAdapter(ViaCepClient client) {
        this.client = client;
    }

    @Override
    public Endereco buscar(String cep) {
        ViaCepDTO dto = client.consultar(cep);
        return new Endereco(
            dto.logradouro(), // rua
            dto.localidade(), // cidade
            dto.uf());        // estado
    }
}
```



```
public class ClienteService {  
    private final BuscadorDeEndereco buscador;  
  
    public ClienteService(BuscadorDeEndereco buscador) {  
        this.buscador = buscador;  
    }  
  
    public void cadastrar(String cep) {  
        Endereco endereco = buscador.buscar(cep);  
        // dominio trabalha apenas com Endereco  
    }  
}
```

Resultado

Trocar ViaCEP por BrasilAPI exige outro Adapter; ClienteService permanece igual.



Adapter: razão de existir

Use quando algo funciona, mas não fala a língua que o sistema espera.

- Resolve incompatibilidade sem reescrever as partes.
- Preserva classes existentes e APIs externas.
- Evita espalhar DTOs externos pelo domínio.
- Facilita testes, troca de fornecedor e manutenção.

Por que estrutural?

Acrescenta uma peça de encaixe à estrutura de colaboração entre objetos.

Decorator



Analogia: café em camadas

O café base continua sendo café. Podemos adicionar leite, chocolate ou chantilly sem mudar a receita original.

- Café simples: comportamento base.
- Leite e chocolate: camadas adicionais.
- Cada camada envolve a anterior.

No código

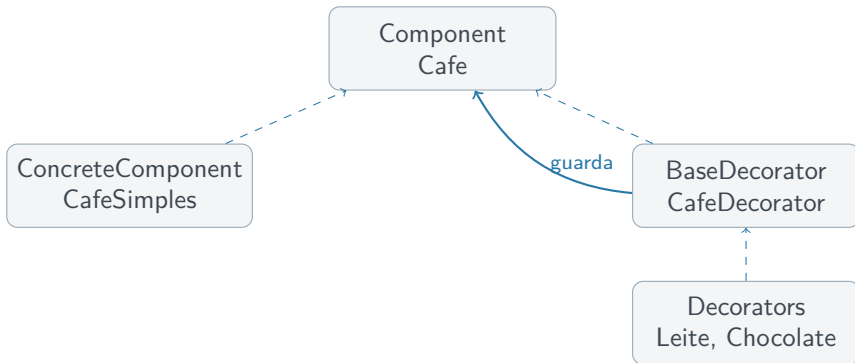
O objeto recebe responsabilidades extras mantendo a mesma interface.





```
class ProdutoService {
    Produto buscar(Long id) {
        log.info("buscando...");
        if (cache.containsKey(id)) {
            return cache.get(id);
        }
        Produto p = repository.findById(id);
        cache.put(id, p);
        log.info("encontrado");
        return p;
    }
}
```

Regra de negócio, log e cache estão misturados. Adicionar retry aumenta ainda mais o acoplamento.



Regra de ouro

O Decorator implementa a mesma interface e delega para o objeto decorado.



```
public interface Cafe {  
    String descricao();  
    double preco();  
}  
  
public class CafeSimples implements Cafe {  
    public String descricao() {  
        return "Cafe simples";  
    }  
  
    public double preco() {  
        return 5.0;  
    }  
}
```

CafeSimples contém somente o comportamento essencial e desconhece os adicionais.



```
public abstract class CafeDecorator implements Cafe {  
    protected final Cafe cafe;  
  
    protected CafeDecorator(Cafe cafe) {  
        this.cafe = cafe;  
    }  
}
```

- Implementa o mesmo contrato Cafe.
- Guarda uma referência para outro Cafe.
- Oferece a base comum às camadas concretas.



```
public class LeiteDecorator extends CafeDecorator {  
    public LeiteDecorator(Cafe cafe) {  
        super(cafe);  
    }  
  
    public String descricao() {  
        return cafe.descricao() + ", leite";  
    }  
  
    public double preco() {  
        return cafe.preco() + 2.0;  
    }  
}
```

A camada delega ao objeto interno e acrescenta sua responsabilidade.



```
public class ChocolateDecorator extends CafeDecorator {  
    public ChocolateDecorator(Cafe cafe) {  
        super(cafe);  
    }  
  
    public String descricao() {  
        return cafe.descricao() + ", chocolate";  
    }  
  
    public double preco() {  
        return cafe.preco() + 3.0;  
    }  
}
```



```
Cafe cafe = new CafeSimples();  
cafe = new LeiteDecorator(cafe);  
cafe = new ChocolateDecorator(cafe);  
  
System.out.println(cafe.descricao());  
System.out.println(cafe.preco());
```

Saída

```
Cafe simples, leite, chocolate  
10.0
```

Para remover chocolate, basta não aplicar essa camada. Nenhuma classe existente muda.

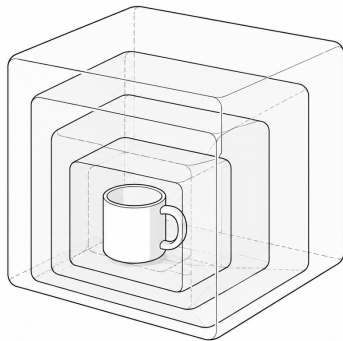


Sem Decorator, cada combinação pode virar uma classe:

- CafeComLeite
- CafeComChocolate
- CafeComLeiteEChocolate
- CafeComLeiteChocolateEChantilly

Composição flexível

O Decorator troca herança rígida por camadas combinadas em tempo de execução.



Comparação e prática



Critério	Adapter	Decorator
Problema	Interfaces incompatíveis	Adicionar responsabilidades
Estrutura	Ponte entre mundos	Camadas no mesmo contrato
Interface final	Converte para outra	Mantém a mesma
Exemplo	ViaCepAdapter	LeiteDecorator
Pergunta	Como faço encaixar?	Como adiciono sem alterar?

Resumo

Adapter traduz. Decorator envolve.



Módulo 1 – Adapter Crie `ConversorDeMoeda` e adapte `BancoCentralClient` e `ExchangeRateClient`.

Módulo 2 – Decorator Adicione `AuditoriaDecorator` e `CacheDecorator` sobre o conversor.

Módulo 3 – Teste Mostre que o service usa apenas `ConversorDeMoeda` e desconhece as APIs externas.

Entrega esperada

Diagrama simples, classes Java principais e explicação de onde cada padrão aparece.



Adapter faz interfaces incompatíveis trabalharem juntas e protege o domínio de APIs externas ou legadas.

Decorator adiciona responsabilidades dinamicamente, mantém a interface e evita explosão de subclasses.

Estruturais resolvem problemas de encaixe, composição e organização entre objetos.

Referência: Gamma et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.